v-bind

v-bind作用：　　动态地绑定一个或多个属性，或者一个组件prop到表达式。

语法：　　在元素上 v-bind:属性名="常量||变量名"

简写形式：:属性名="变量名"

```
<div id="app">
        <a v-bind:href="uhref">//动态绑定属性
           <img :src="url" >//v-bind的一个简写，省略v-bind，用冒号代替：
        </a>
        <ul>
           <li v-for="item in  imgurl">
               <img :src="'{{}}'" >//错误写法：  不能使用{{}} 语法
              <img :src="item" >
           </li>
        </ul>
</div>
<script>
var vm =new Vue({
  el:"#app",
  data:{
   uhref:"https://www.baidu.com",
    url:"https://www.baidu.com/img/PCtm_d9c8750bed0b3c7d089fa7d55720d6cf.png",

    imgurl:[
          "https://www-static.chinacdn.starbucks.com.cn/prod/images/pages/tmall-2020-06-29-zh.jpg",
          "https://www-static.chinacdn.starbucks.com.cn/prod/images/pages/starbucks-design-studio-web-china.jpg",
          "https://www-static.chinacdn.starbucks.com.cn/prod/images/pages/homepage-career-cn.jpg",
        ]
      }
  })
</script>
```

普通class

使用数组

数组中使用对象代替三元表达式

数组中使用三元表达式

使用对象

对象语法

动态绑定class

Vue样式绑定

数组样式

对象内联样式

```
<style>
        .red{color: red; }
        .italic{font-style: italic; }
         .bor{ border: 1px solid blue; }
         .blue{color: blue;}
</style>
<div id="app">
            <h2 class="red italic">普通class</h2>
            <!-- 动态绑定class，使用数组来罗列多个绑定的类名 -->
            <h2 :class="['red','italic','blue']">使用数组</h2>
            <!-- 数组对象语法 -->
            <h2 :class="['red',{'bor':flag}]">数组中使用对象代替三元表达式</h2>
            <h2 :class="[flag ? 'italic' : 'red']">数组中使用三元表达式</h2>
            <h2 :class="classobj">使用对象</h2>
            <!-- v-bind:class指令可以与普通class共存 -->
            <h3 :class="{bor:true,italic:flag}">对象语法</h3>
</div>
<script>
   var vm=new Vue({
              el:"#app",
              data:{
                  flag:true,
                  classobj:{
                      red:true,
                      italic:true,
                      bor:false,
                      blue:true
                  }
              }
          })
</script>
```

动态绑定style

```
<div id="app">
            <!-- 动态绑定 style 的对象语法，和css非常相似  其实他就是一个JavaScript对象--
```

```
>
        <div :style="{color:'green'}">Vue样式绑定</div>
        <h2  :style="[sobj1,sobj2]">数组样式</h2>

        <span :style="{color:'red',fontSize:fz+'px'}">对象内联样式</span>
</div>
<script type="text/javascript">
        var vm=new Vue({
            el:"#app",
            data:{
                sobj1:{
                    color:"pink"
                },
                sobj2:{
                    /* css属性名要驼峰命名(fontSize) ,或者短横命名(font-size) */
                    // fontSize:"50px"
                    "font-size":"50px"
                },
                fz:10
            }
        })
 </script>
```

v-on

绑定一个事件或多个事件，事件函数的参数名代表了在方法对象methods中所定义的函数。

语法：v-on：事件名=”方法名 ”

简写：@ 事件名=”方法名”

```
<div id="app">
        <span>点击的次数：  {{num}}</span>
        <!-- click 的值 是表达式， 我们可以动态拿到vue，data属性中的数据然后进行操作-->
        <button type="button" v-on:click="num++">+1</button>
        <!--dblclick 双击  -->
        <button type="button" v-on:dblclick="num++">双击</button>
        <button type="button" @click="clicktwo()">只能+2次</button>
        <p>{{msg}}</p>
        <!-- 不传参的时候可以不写圆括号,直接写函数名 -->
        <button @click="rString" type="button">反转</button>
        <!-- 使用{{}} 调用方法必须写圆括号 -->
        <span>{{testString()}}</span>

        //传参
        //不加括号绑定事件，默认情况下第一个形参就是事件对象
        <button type="button" @click="test">没加括号</button>
        //加了括号，无论你传不传实参，形参默认就是要接收你传递的实参
        <button type="button" @click="test()">加了括号</button>
        <button type="button" @click="test(123)">123</button>
```

```
        //绑定事件，手动传递事件对象,$event就是vue框架专门指定像事件对象的关键词，
        //$event就是事件对象
        <button type="button" @click="rev($event,'手动传递事件对象')">点击
</button>
</div>
<script type="text/javascript">
var vm=new Vue({
            el:"#app",
            data:{num:0,msg:"abcdefg"},
            methods:{//
                // methods中是用来存放函数，这里面的函数会自动成为vue实例的方法
                testString:function(){
                    var hobby=["吃饭","睡觉","打豆豆"]
                    return hobby;
                },
                // 限制最大点击次数
                clicktwo:function(){
                    console.log(this)
                    console.log(this==vm)//true
                    var n= vm.num;
                    vm.num=Math.min(++n,2);
                },
                rString(){
                    // 将字符反转   分割，反转，拼接
                        this.msg=this.msg.split('').reverse().join(' ')
                },
                test(ev){
                    console.log(ev)//事件对象
                },
                rev(ev,str){
                    console.log("ev",ev)
                    console.log(str);
                }
            }
        })
 </script>
```

v-on 绑定多个事件

```
<div id="app">
        <!--绑定多个事件  -->
        <button type="button" v-on="{click:click,dblclick:dbclick}">绑定了多个事件
</button>
        <button type="button" v-on="{click:str,dblclick:math}">绑定了多个事件
</button>
</div>
 <script type="text/javascript">
 var vm=new Vue({
            el:"#app",
```

```
            data:{num:0,msg:"abcdefg"},
            methods:{
                click(){ console.log("点击事件触发了")
                },
                dbclick(){ console.log("双击");
                },
                str(){  console.log('字符串')
                },
                math(){console.log('数字') }
            }
        })
 </script>
```

键盘修饰符

这些修饰符是便捷一些按键操作的，比如要判断按键的键值的时候

.keyCode ：keyCode代表的是键值的数值，只有当触发事件的键值相同的时候才会触发事件

键值修饰符的一些特殊的别名

.enter 回车

.tab tab键

.delete 删除”和“退格

.esc

.space

.up ↑

.down ↓

.left ←

.right →

```
      <div id="app">
         <!-- 键盘事件 -->
         <input type="text" @keyup.once="show($event)" />
         <p>键盘事件@keydown</p>
         <input type="text" @keydown="show($event)" />
         <input type="text" @keydown.enter="show($event)" /><!-- .enter -->
         <input type="text" @keydown.space="show($event)" /><!-- .space -->
      </div>
<script type="text/javascript">
     var vm=new Vue({
       el:"#app",
        methods:{
             show(e){
              console.log(e.keyCode)
              console.log(e)
```

```
        }
      }
    })
</script>
```

通用修饰符

事件指令提供了很多的修饰符，可以快捷的阻止默认事件，阻止冒泡等等，通用修饰符：这些修饰符是所有事件都通用的

.stop 阻止事件冒泡（调用 event.stopPropagation()）

.prevent 阻止默认事件（调用 event.preventDefault()。）

.capture 添加事件捕获

.self 只能由自身触发事件

.once 该事件只会触发一次